

Occupancy Mapping of Indoor Environment Using P3DX Robot and Ultrasonic Sensors in CoppeliaSim

Author(s): Muhammad Bari Akmal
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Robot Otonom
YouTube video link: <https://youtu.be/XYKgtL1H1vo>
GitHub link: <https://github.com/Drimal-835/Otomatic-of-Robotic-System>

Pada tugas ini, digunakan model robot P3DX sebagai objek simulasi dan CoppeliaSim sebagai aplikasi untuk melakukan simulasi. Tujuan dari tugas ini adalah melakukan pemetaan (*mapping*) lingkungan dalam ruangan menggunakan empat sensor ultrasonik yang terpasang pada robot P3DX. Robot bergerak secara otomatis menggunakan algoritma Braitenberg bawaan CoppeliaSim, sementara kode Python membaca data sensor dan mencatat titik-titik hambatan ke dalam frame dunia (*world frame*). Peta akan ditampilkan setelah simulasi selesai.

Lingkungan Simulasi (Actual Map)

Gambar 1 menunjukkan tampilan atas lingkungan simulasi yang digunakan dalam CoppeliaSim. Lingkungan ini merupakan ruang kelas B202 yang berisi meja, kursi, dan dinding pembatas.

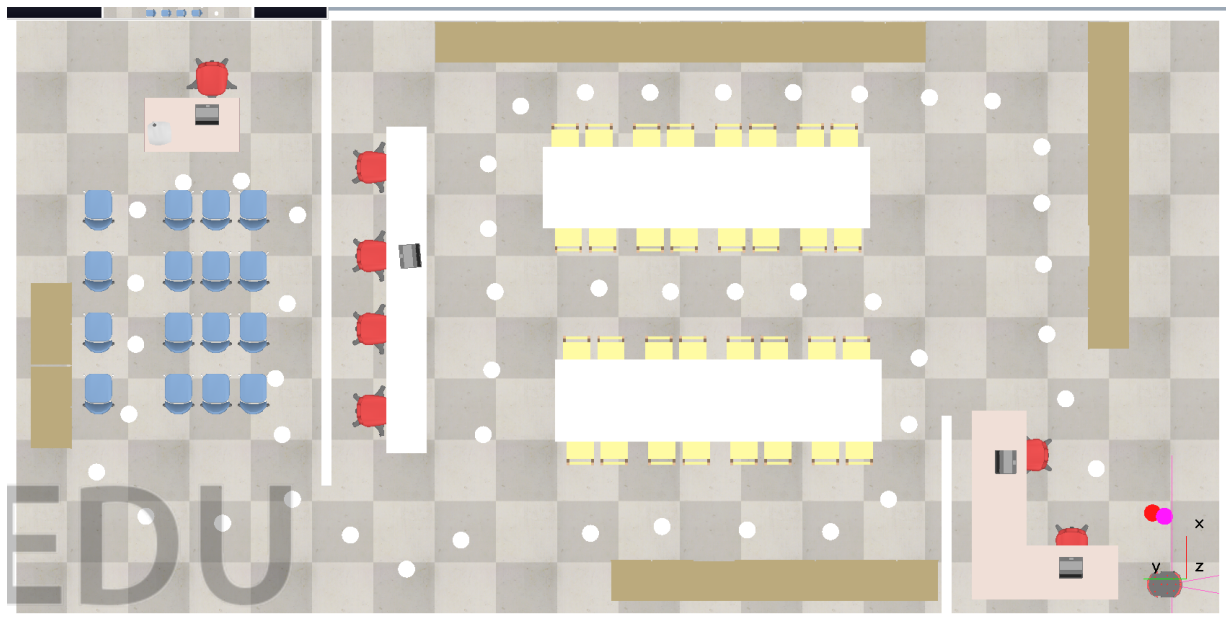


Figure 1: Tampilan atas lingkungan simulasi B202 di CoppeliaSim (Actual Map)

Hasil Pemetaan (Mapping Result)

Gambar 2 menunjukkan hasil pemetaan yang diperoleh dari pembacaan keempat sensor ultrasonik (indeks 0, 3, 4, dan 7) selama robot bergerak mengelilingi lingkungan. Setiap titik pada grafik merepresentasikan satu deteksi hambatan yang telah ditransformasikan ke dalam frame dunia menggunakan matriks transformasi homogen.

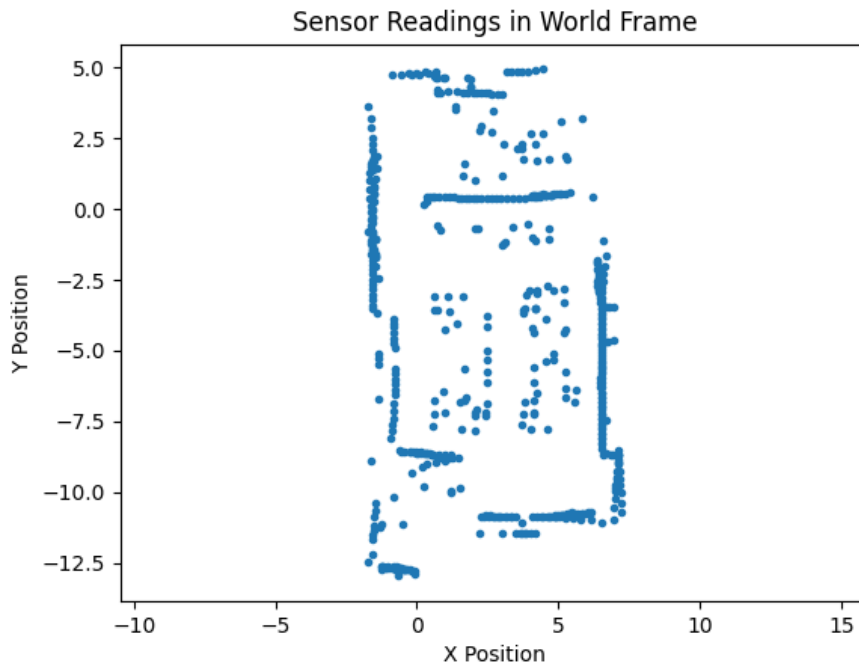


Figure 2: Hasil pemetaan sensor ultrasonik P3DX dalam world frame

Kode Python

Berikut adalah kode Python yang digunakan untuk melakukan pemetaan menggunakan empat sensor ultrasonik (indeks 0, 3, 4, dan 7). Kode ini terhubung ke CoppeliaSim melalui ZMQ Remote API, membaca posisi robot dan titik deteksi sensor, mentransformasikan setiap titik ke dalam frame dunia, dan menampilkan peta setelah simulasi berhenti.

```

1  import time
2  import math
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from coppeliasim_zmqremoteapi_client import RemoteAPIClient
6
7
8  def transformMat(alpha, beta, gamma, tx, ty, tz):
9      rotx = np.array([
10         [1, 0, 0],
11         [0, math.cos(alpha), -math.sin(alpha)],
12         [0, math.sin(alpha), math.cos(alpha)]
13     ])
14      roty = np.array([
15         [math.cos(beta), 0, math.sin(beta)],
16         [0, 1, 0],
17         [-math.sin(beta), 0, math.cos(beta)]
18     ])
19      rotz = np.array([
20         [math.cos(gamma), -math.sin(gamma), 0],
21         [math.sin(gamma), math.cos(gamma), 0],
22         [0, 0, 1]
23     ])
24      rot_total = np.matmul(np.matmul(rotx, roty), rotz)
25      trans_vector = np.array([[tx], [ty], [tz]])
26      R_t_3x4 = np.hstack((rot_total, trans_vector))
27      homogeneous_row = np.array([[0, 0, 0, 1]])

```

```

28     return np.vstack((R_t_3x4, homogeneous_row))
29
30
31 # 1. Connect
32 client = RemoteAPIClient()
33 sim = client.require('sim')
34
35 # 2. Start simulation
36 sim.startSimulation()
37 print("Simulation started.")
38 sim.addLog(1, "Mapping started from Python!")
39
40 # 3. Get handles
41 p3dx = sim.getObject("/PioneerP3DX")
42
43 SENSOR_INDICES = [0, 3, 4, 7]    # 0-based indices matching the Lua script
44 sensor_handles = []
45 for idx in SENSOR_INDICES:
46     h = sim.getObject("/ultrasonicSensor", {"index": idx})
47     sensor_handles.append(h)
48
49 # 4. Storage
50 hits_x = {idx: [] for idx in SENSOR_INDICES}
51 hits_y = {idx: [] for idx in SENSOR_INDICES}
52 path_x, path_y = [], []
53
54 # 5. Main loop
55 RUN_SECONDS = 60
56
57 try:
58     start_time = time.time()
59     step = 0
60     while (time.time() - start_time) < RUN_SECONDS:
61         elapsed = time.time() - start_time
62         print(f"  t={elapsed:.1f}s", end="\r")
63
64         # Robot pose in world frame
65         p3dx_pos = sim.getObjectPosition(p3dx, sim.handle_world)
66         p3dx_ori = sim.getObjectOrientation(p3dx, sim.handle_world)
67         path_x.append(p3dx_pos[0])
68         path_y.append(p3dx_pos[1])
69
70         T_body_world = transformMat(
71             p3dx_ori[0], p3dx_ori[1], p3dx_ori[2],
72             p3dx_pos[0], p3dx_pos[1], p3dx_pos[2]
73         )
74
75         for i, (idx, h) in enumerate(zip(SENSOR_INDICES, sensor_handles)):
76             res, dist, prox_point, prox_obj, prox_n = \
77                 sim.readProximitySensor(h)
78
79             if res and dist > 0:
80                 # Use prox_point: actual hit XYZ in sensor local frame
81                 sensor_hit = np.array([
82                     prox_point[0],
83                     prox_point[1],
84                     prox_point[2],
85                     1])
86             ])
87
88             # Sensor pose relative to robot body
89             s_pos = sim.getObjectPosition(h, p3dx)

```

```

90         s_ori = sim.getObjectOrientation(h, p3dx)
91         T_sensor_body = transformMat(
92             s_ori[0], s_ori[1], s_ori[2],
93             s_pos[0], s_pos[1], s_pos[2]
94         )
95
96         # sensor frame -> body frame -> world frame
97         hit_body = np.matmul(T_sensor_body, sensor_hit)
98         hit_world = np.matmul(T_body_world, hit_body)
99
100         hits_x[idx].append(hit_world[0][0])
101         hits_y[idx].append(hit_world[1][0])
102
103         step += 1
104         time.sleep(0.02)
105
106     finally:
107         sim.stopSimulation()
108         print("\nSimulation stopped.")
109
110     # 6. Plot
111     all_x = []
112     all_y = []
113     for idx in SENSOR_INDICES:
114         all_x.extend(hits_x[idx])
115         all_y.extend(hits_y[idx])
116
117     plt.figure()
118     plt.plot(all_x, all_y, '.')
119     plt.xlabel('X Position')
120     plt.ylabel('Y Position')
121     plt.title('Sensor Readings in World Frame')
122     plt.axis('equal')
123     plt.show()

```

Penjelasan Algoritma

Algoritma pemetaan yang digunakan bekerja melalui tiga tahap transformasi koordinat:

1. **Frame sensor ke frame robot (body):** Titik deteksi yang dikembalikan oleh `sim.readProximitySensor()` dalam koordinat lokal sensor ditransformasikan ke koordinat robot menggunakan matriks transformasi homogen $T_{\text{sensor} \rightarrow \text{body}}$.
2. **Frame robot ke frame dunia (world):** Hasil transformasi pertama kemudian ditransformasikan lagi ke frame dunia menggunakan $T_{\text{body} \rightarrow \text{world}}$ yang dibangun dari posisi dan orientasi robot aktual pada setiap langkah waktu.
3. **Akumulasi titik:** Semua titik hasil transformasi disimpan dan divisualisasikan setelah simulasi selesai menggunakan `matplotlib`.

Sensor yang digunakan adalah sensor ultrasonik indeks 0, 3, 4, dan 7 (berbasis indeks 0 sesuai skrip Lua bawaan CoppeliaSim), yang masing-masing menghadap ke arah depan-kiri, kiri, kanan, dan depan-kanan robot. Pergerakan robot sepenuhnya dikendalikan oleh algoritma Braitenberg pada skrip Lua bawaan CoppeliaSim tanpa modifikasi dari sisi Python.